



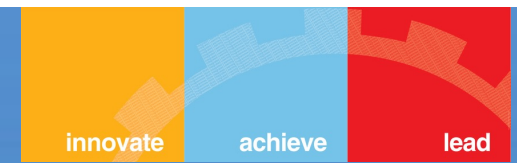
BITS Pilani

Conversational AI

Model Landscape & Cost Engineering

Dr Bharathi R

Adjunct Professor, CSIS WILP BITS PILANI



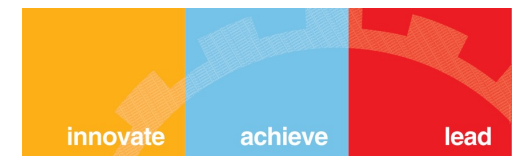
BITS Pilani



AIMLCZG521 , Conversational AI

Lecture No. 4 | Module 1

Disclaimer & Acknowledgement



DISCLAIMER

These instructional materials have been developed by the instructor for educational purposes. All content is intended solely to support the learning objectives of this course. While every effort has been made to ensure accuracy and completeness, learners are encouraged to consult original source publications for authoritative reference.

ACKNOWLEDGEMENT

These instructional materials have been developed by the instructor with due acknowledgement of the original publication authors and the broader academic community for their openly accessible resources, along with special recognition of Prof. Bhagath's Conv-AI-WILP (October 2025) presentation materials, which have been adapted and modified to support the learning objectives of this course.

Learning Objectives



Navigate the LLM Ecosystem

Map 2025/2026 production architectures — Dense, MoE, SLM, SSM — with quantitative benchmarks (MMLU, context length, cost/1M tokens)



Master Quantization

Apply GPTQ / AWQ / QLoRA to compress 70B models from 140 GB → 36 GB with <2% quality loss



Calculate GPU Memory

Use $M_{\text{total}} = M_{\text{weights}} + M_{\text{KV}} + M_{\text{activation}}$ to budget VRAM for any model configuration



Optimise API Cost

Design routing + caching strategies achieving 70–90% cost reduction in production deployments



Deploy Intelligently

Decide between managed API and self-hosted vLLM using break-even analysis at token-volume thresholds

Agenda



Part 1 · Model Landscape

- ▶ Dense Transformers — GPT-4o, Claude 3.7, Llama 3.3, Gemini 2.0
- ▶ Mixture-of-Experts — Mixtral 8×22B, DeepSeek-V3, Qwen2-MoE
- ▶ Small Language Models — Phi-4, Gemma 3, Qwen2.5, Llama 3.2
- ▶ State Space Models — Mamba-2, Jamba-1.5 (hybrid)
- ▶ Intelligent Model Routing & Cost-Complexity Trade-offs

Part 2 · Cost Engineering

- ▶ Quantization: FP32→NF4, GPTQ, AWQ, FP8 (H100 native)
- ▶ LoRA & QLoRA Fine-Tuning — 31× memory reduction
- ▶ GPU Memory Estimation: Weights + KV-Cache + Activations
- ▶ Token Pricing, Prompt Caching — up to 90% cost reduction
- ▶ Production Optimization Playbook & Self-Hosting Break-Even

The 2025 LLM Landscape



Production-grade architectures — benchmarks, context windows, and cost profiles

Architecture	Key Models (2025)	Context Window	MMLU	Cost / 1M Tokens	Best Use Case
Dense Transformer	GPT-4o, Claude 3.7 Sonnet, Llama 3.3 70B, Gemini 2.0 Pro	128K – 1M	86–90%	\$3 – \$30	Complex reasoning, multimodal
Mixture-of-Experts	DeepSeek-V3, Mixtral 8×22B, Qwen2-57B-A14B, Jamba-1.5	32K – 256K	80–88%	\$0.3 – \$2	Cost-effective at scale
Small LMs (1–14B)	Phi-4, Gemma 3, Qwen2.5-7B, Llama 3.2-3B	4K – 128K	72–85%	\$0.05 – \$0.3	Routing, edge, classification
State Space Models	Mamba-2, RWKV-6, Jamba-1.5 (hybrid)	1M+	68–75%	Variable	Ultra-long context tasks

Sources: MMLU from Open LLM Leaderboard (HuggingFace), May 2025. Pricing indicative — verify at vendor documentation before production use.



The 2026 LLM Landscape

Production-grade architectures — benchmarks, context windows, and cost profiles

ARCHITECTURE	KEY MODELS (2026)	CONTEXT WINDOW	MMLU	COST / 1M TOKENS	BEST USE CASE
Dense Transformer	GPT-5, Claude 4, Llama 4.0, Gemini 3.0 Pro Emerging multimodal	256K – 2M	92–96%	\$2 – \$15	Reasoning, analysis, document processing
Mixture-of-Experts	DeepSeek-V4, Mixtral 8x70B, Qwen3-MoE, Grok-2 Scaling efficiency	64K – 512K	88–92%	\$0.2 – \$0.8	Production APIs, multi-task pipelines
Small LMs (1–15B)	Phi-5, Gemma 4, Qwen3-4B, Llama 3.5-7B On-device ready	8K – 64K	78–86%	\$0.001 – \$0.05	Edge, mobile, local inference
State Space Models (Hybrid)	Mamba-3, RWKV-7, Jamba 2.0 (hybrid), Liquid-2 Linear scaling	32K – 1M+	82–88%	Variable	Long-context streaming, inference-optimized

Source: Projected trends from 2025 LLM Landscape — industry trajectories, pricing evolution, and emerging architectures. Benchmarks are estimated based on 2025 baseline performance improvements. Cost and availability subject to market dynamics.



Dense Transformer Models

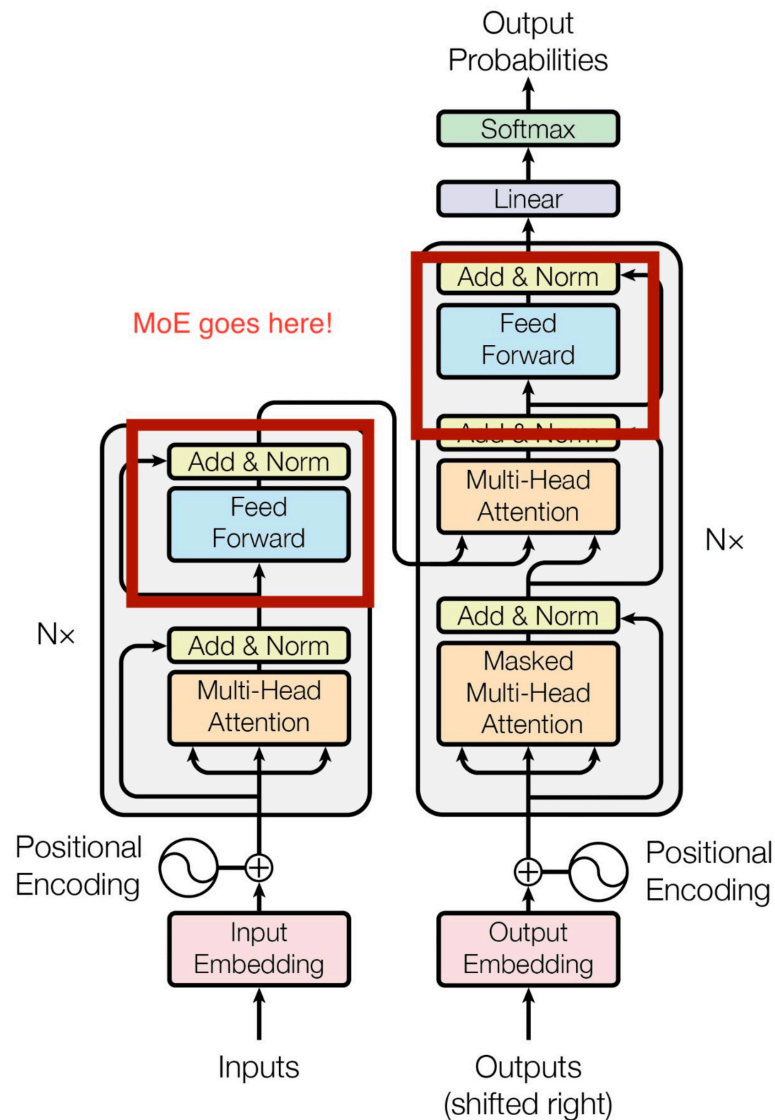
State-of-the-Art Foundation Models — 2025 Production Landscape

GPT-4o (OpenAI)	CTX: 128K	MMLU: 88.7%	Multimodal: text + vision + audio. Native tool-use & function calling. Batch API for 50% cost cut. [1]
Claude 3.7 Sonnet	CTX: 200K	MMLU: 90.1%	Anthropic's reasoning flagship. Extended Thinking mode (+32K CoT tokens). Best-in-class coding & analysis. [2]
Llama 3.3 70B (Meta)	CTX: 128K	MMLU: 86.0%	Open-weights. Rivals GPT-4 Turbo on MMLU. Self-hostable on vLLM at ~\$0.10–0.30/1M tokens. [3]
Gemini 2.0 Flash	CTX: 1M	MMLU: 87.0%	Google's multimodal production model. Cheapest at \$0.075/1M input. Native grounding via Search. [4]

Key Characteristic: All parameters active per token → consistent quality at higher FLOP cost. 70B BF16 ≈ 140 GB VRAM → typically requires 2× H100 80GB for single-model inference.

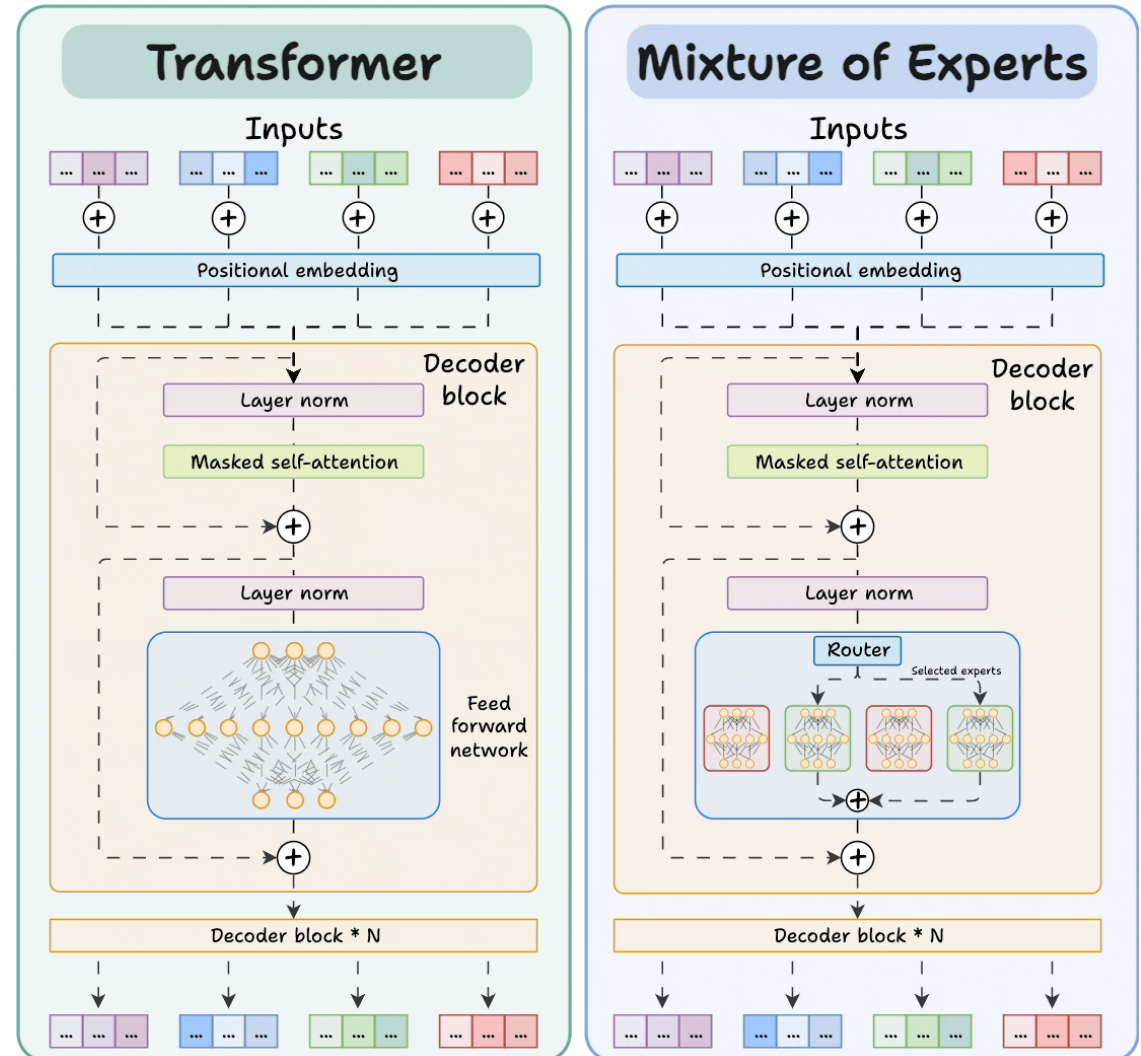
[1] OpenAI GPT-4o System Card 2024 [2] Anthropic Claude 3.7 Model Card Mar 2025 [3] Meta Llama 3.3 Technical Report Dec 2024 [4] Google Gemini 2.0 Tech Spec Feb 2025

Mixture of Experts

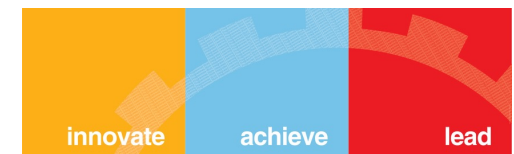


Transformer vs. Mixture of Experts

[join.DailyDoseofDS.com](https://www.dailydoseofds.com)



Mixture of Experts: Router Mechanism



How Sparse Activation Achieves Efficiency

The Routing Decision

$\text{router_logits} = W_{\text{router}} \cdot x$

$\text{top_k_experts} = \text{TopK}(\text{router_logits}, k=2)$

$\text{weights} = \text{softmax}(\text{top_k_logits})$

$\text{output} = \sum (w_i \times \text{Expert}_i(x))$

Example: DeepSeek-V3 (2025)

- 671B total parameters (256 experts $\times$ 2.5B each)
- Router selects top-2 experts per token
- Active params per token: 37B (94.5% saving)
- API cost: $\sim$ 5% of GPT-4o equivalent

Router Network

Input Token [d dimensions]

Router Linear Layer [d $\rightarrow$ num_experts]

Select Top-K (typically K=2)

Load Balancing

Challenge: Expert Collapse

Router favours certain experts; $\sim$ 15% of naive training runs collapse to 1–2 hot experts.

Solution: Auxiliary Load-Balance Loss

$L_{\text{aux}} = \alpha \times \text{load_balance_loss}$

DeepSeek-V3 uses $\alpha = 0.001$; encourages uniform token distribution.

Ref: Shazeer et al. (2017) Sparsely-Gated MoE NeurIPS; DeepSeek-AI (2024) DeepSeek-V3 Technical Report arXiv:2412.19437

Mixture of Experts Models

Cost-Effective Scaling — 2025 Production Models

DeepSeek-V3	Total: 671B	Active: 37B	Saving: 94.5%	Best open-source MoE 2025. Rivals GPT-4o. 128K ctx. <\$0.50/1M tokens on API. [5]
Mixtral 8×22B	Total: 141B	Active: 39B	Saving: 72.3%	Strong coder/reasoner. 65K ctx. Available on Mistral API & self-host via vLLM. [6]
Qwen2-72B-72B	Total: 72B	Active: 14B	Saving: 75.4%	Alibaba open-weights. Multilingual (29 langs). Outperforms Llama-3 70B on MMLU. [7]
Jamba-1.5 Large	Total: 398B	Active: 52B	Saving: 86.9%	AI21 hybrid SSM+Attention. 256K ctx. Production API. Best long-doc model. [8]

Efficiency Formula

Cost Reduction = $1 - (\text{Active} / \text{Total})$
 DeepSeek-V3: $1 - 37\text{B}/671\text{B} = 94.5\%$ compute savings

Key Challenges

- All experts must be in VRAM (671B needs ~8× H100)
- Router adds ~5–15% inference latency per token
- Load balancing requires careful α tuning

[5] DeepSeek-AI arXiv:2412.19437 [6] Mistral AI Mixtral-8×22B Apr 2024 [7] Qwen2 Report arXiv:2407.10671 [8] AI21 Labs Jamba-1.5 Aug 2024

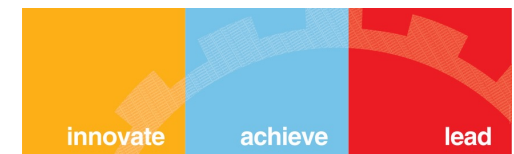
Small Language Models (SLMs)

Specialized Efficient Models — 1–14B Parameters for Edge & Production Routing

Model	Params	Context	MMLU	Strengths	Deployment Target
Phi-4 (Microsoft)	14B	16K	84.8%	Reasoning, coding, math — rivals GPT-4o-mini on most benchmarks	Azure AI, Windows Copilot+, on-device GPU
Gemma 3 (Google)	4B / 27B	128K	73 / 81%	Safety-tuned; multilingual (140 langs); vision capable	On-device Android, Vertex AI, edge TPU
Qwen2.5 (Alibaba)	7B	128K	79.9%	Multilingual (29 langs), strong code & math, Apache-2.0	Local inference, HuggingFace self-host
Llama 3.2 (Meta)	1B / 3B	128K	60 / 75%	Ultra-lightweight; vision variant at 11B; tool-use	Edge devices, IoT, mobile apps
SmolLM2 (HF)	135M – 1.7B	8K	~55%	Runs on CPU; browser & microcontroller deployable	On-device, privacy-first, serverless
Production Tip: Use SLMs as intelligent routers (intent classification, NER, slot-filling) handling ~80% of queries at <5% of GPT-4 cost. A Phi-4-mini router + Claude 3.7 backend cut one enterprise chatbot bill by 78% with no CSAT drop.					

Ref: Microsoft Phi-4 Report Dec 2024; Google Gemma 3 Mar 2025; Qwen2.5 arXiv:2412.15115; Meta Llama 3.2 Sep 2024

Attention Mechanism Fundamentals



Understanding the Core of Transformer Architecture

Self-Attention in Transformers

Query (Q): What am I looking for? — compared against all Keys to produce attention scores

Key (K): What do I contain? — indexed by the attention router to find relevant context

Value (V): What information do I have? — aggregated by attention weights into the output

Scaled Dot-Product Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Step 1: Score

Compute QK^T
Dot product → raw
attention scores $[n \times n]$

Step 2: Scale

Divide by $\sqrt{d_k}$
Prevents vanishing gradients
for large d_k

Step 3: Normalise

Softmax over row
Converts to probability
distribution $[0,1]$

Step 4: Aggregate

Multiply by V
Weighted sum of Values →
context-aware output

Ref: Vaswani et al. (2017) 'Attention Is All You Need' NeurIPS arXiv:1706.03762

Transformer's Quadratic Complexity Problem

Why Attention is Expensive for Long Sequences

The Complexity Challenge

$$O(n^2 \cdot d)$$

n = sequence length, d = model dimension

Memory Requirements

512 tokens	→ $512^2 = 262\text{K}$ attention scores
2K tokens	→ $2,048^2 = 4.2\text{M}$ scores
8K tokens	→ $8,192^2 = 67\text{M}$ scores
128K tokens	→ $128,000^2 = 16.4\text{B}$ scores ⚠ impractical

Why $O(n^2)$?

Every token attends to every other token:

$QK^T = [n \times d] \cdot [d \times n] \rightarrow [n \times n]$ matrix

For L layers: total scores = $L \times n^2$

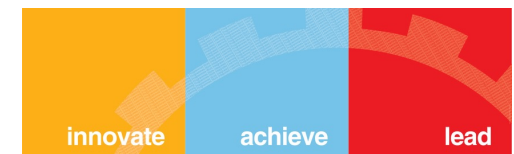
GPT-4 (L=96, n=128K): $96 \times 16.4\text{B} \approx 1.57$ trillion ops

Grows quadratically!

**Quadratic scaling makes naive attention on 100K+ documents impractical.
FlashAttention-3 and Sparse Attention are the production mitigations (2024–2025).**

Ref: Dao et al. (2022) FlashAttention arXiv:2205.14135; Dao (2023) FlashAttention-2 arXiv:2307.08691; Shah et al. (2024) FlashAttention-3 arXiv:2407.08608

State Space Models: Linear Complexity Solution



How SSMs Address the Quadratic Complexity Problem

✗ Transformer Attention

Complexity:	$O(n^2 \cdot d)$
Token visibility:	Every token sees all tokens
Memory (128K seq):	~16 GB attention map
Context scaling:	Impractical > 128K tokens

✓ State Space Models (Mamba-2)

Complexity:	$O(n \cdot d)$
Token visibility:	Sequential state propagation
Memory:	Constant per step, $O(d_{\text{state}})$
Context scaling:	1M+ tokens efficiently

State Space Representation — Mamba / S4 / Mamba-2

$$h_t = A \cdot h_{t-1} + B \cdot x_t$$
$$y_t = C \cdot h_t + D \cdot x_t$$

- h_t Hidden state — fixed-size compressed history ($d_{\text{state}}=16$ in Mamba-2)
- x_t Current input token embedding (model dimension d)
- A, B, C, D Learned projections; Mamba-2 makes A, B, C input-dependent (Selective SSM)

🎯 **Key Insight:** Fixed-size state regardless of sequence length → $O(n)$ time & $O(1)$ memory per step → 1M+ token context is practical.

Ref: Gu & Dao (2023) Mamba arXiv:2312.00752; Dao & Gu (2024) Mamba-2 arXiv:2405.21060

State Space Models (SSMs)

Why SSMs?

► **Transformer:** Quadratic cost — 128K context on GPT-4 consumes ~16 GB attention map per layer

► **SSM (Mamba-2):** Linear cost — same 128K context at $O(n)$ — ~10× cheaper in memory

Complexity Comparison

Transformer $O(n^2)$

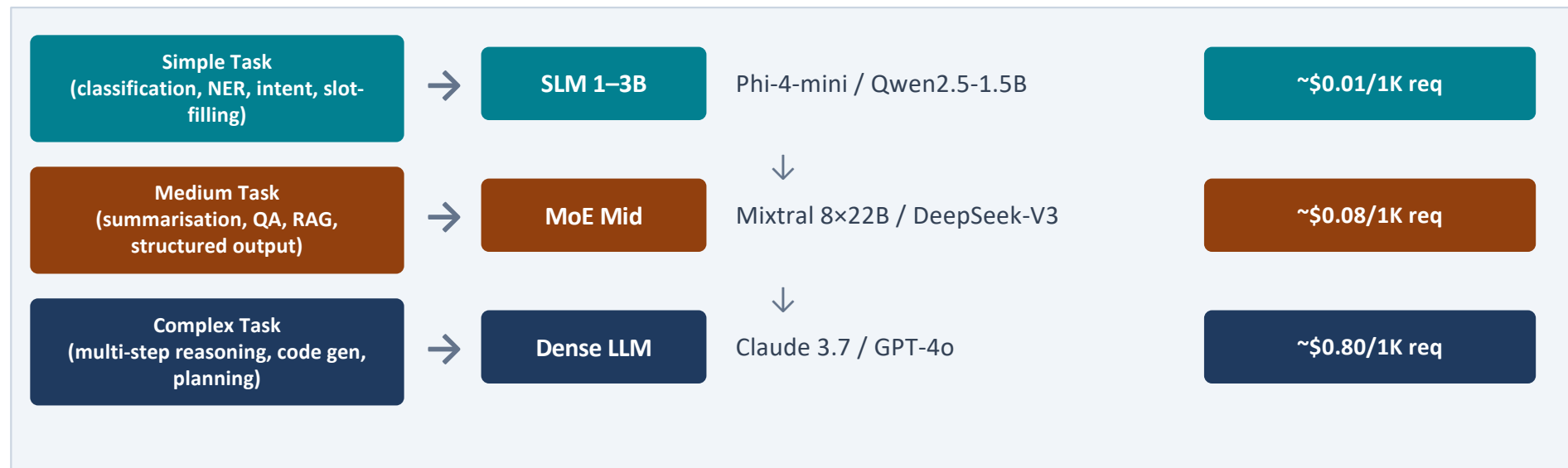
Mamba-2 $O(n)$

Use Case: Processing entire codebases (1M+ tokens), legal document review, long-form RAG without chunking.

Limitations — 2025 Status

- **In-Context Learning:** Weaker at random-access tasks (multi-hop QA, retrieval) — SSMs compress history lossily
- **Production Maturity:** Limited framework support; vLLM & TRT-LLM SSM backends are early-stage
- **Hybrid is Winning:** Jamba-1.5 = 50% Mamba + 50% Attention — captures both efficiency and sharp retrieval
- **Benchmark Gap:** SSMs trail 5–10 MMLU points vs same-size dense transformers on reasoning-heavy tasks

Intelligent Model Routing



Case Study — Enterprise Customer Support Bot (2024)

Architecture: Qwen2.5-1.5B (intent router) → Mixtral 8×22B (FAQ & summarisation) → GPT-4o (edge cases)

Traffic split: 80% SLM / 15% MoE / 5% Dense → Effective cost: \$0.04/1K req vs \$0.80/1K flat GPT-4o

Outcome: 95% cost reduction · CSAT maintained · P50 latency improved 40% vs GPT-4o-only

Ref: Ong et al. (2024) RouteLLM: Learning to Route LLMs with Preference Data arXiv:2406.18665

**We've seen how models differ
in architecture and scaling.**

**Now — let's quantify
how these choices impact cost.**

- Architecture → VRAM requirements
- Quantization → memory vs quality trade-off
- Token pricing → caching & routing savings

Quantization: Making Models Efficient

Reducing Precision Without Sacrificing Quality — 2025 Production Formats

Precision	Bits/Param	Memory (7B)	Memory (70B)	Quality	Speed	Best Use
FP32	32	28 GB	280 GB	100%	1×	Training from scratch
BF16	16	14 GB	140 GB	~99.9%	1.5–2×	LLM inference standard (H100, A100)
FP8 (E4M3/E5M2)	8	7 GB	70 GB	~99.5%	2–3×	H100 native (Transformer Engine) — 2025 default
INT8 (LLM.int8)	8	7 GB	70 GB	~99%	2–4×	Production inference on A10G, V100, A100
NF4 / GPTQ-4bit	4	3.5 GB	35 GB	~98%	3–6×	QLoRA fine-tuning & edge deployment
INT4 / AWQ	4	3.5 GB	35 GB	~98.5%	4–6×	Mobile/edge; best quality at 4-bit



Production Rule: INT8 provides optimal balance – 4x memory reduction, 2-4x speed, with less than 1% quality loss

Ref: Dettmers et al. (2022) LLM.int8() arXiv:2208.07339; Micikevicius et al. (2022) FP8 arXiv:2209.05433; NVIDIA Transformer Engine Docs (2024)

Floating Point Formats

Understanding Precision Trade-offs — Why BF16 Won the LLM Era

FP32 (Float32)

Sign: 1	Exp:8	Mantissa:23
------------	-------	-------------

- Standard for pre-training
- Range: $\pm 3.4 \times 10^{38}$ | 4 bytes/weight
- 70B model = 280 GB

FP16 (Float16)

Sign: 1	Exp:5	Mantissa:10
------------	-------	-------------

- Max value $\pm 65,504 \rightarrow$ overflow risk
- Mixed-precision training (with loss scaling)
- 2 bytes but narrower range than BF16

BF16 (BFloat16)

Sign: 1	Exp:8	Mantissa:7
------------	-------	------------

- Same exponent range as FP32 (no overflow risk)
- Preferred LLM inference — H100, A100
- 2 bytes/weight; 70B = 140 GB

FP8 — 2025 H100 Production Standard

E4M3 (Training): 4-bit exponent · 3-bit mantissa. Max 448.
Higher precision, less range. Better for forward pass.

E5M2 (Inference): 5-bit exponent · 2-bit mantissa. Max 57,344.
Wider range, suited for attention & output layers.

FP8 Transformer Engine doubles throughput vs BF16 on H100 SXM5

🎯 **Sweet Spot: BF16 for all modern GPU inference; FP8 where H100 Tensor Core is available; NF4 for QLoRA or consumer GPU.**

Ref: Micikevicius et al. (2022) 'FP8 Formats for Deep Learning' arXiv:2209.05433; NVIDIA H100 Datasheet (2023)

Post-Training Quantization (PTQ)

Quantize Pre-trained Models Without Retraining — Production Algorithms 2025

GPTQ

Layer-wise Optimal Quantization

- ▶ Hessian-based layer-wise reconstruction
- ▶ Minimises per-layer quantization error
- ▶ 4-bit with ~1% accuracy loss on LLaMA-2 70B
- ▶ Supports 2/3/4/8-bit; GPU-optimised (exllama v2)
- ▶ Throughput: 2–3× vs BF16 on A100 at 4-bit

AWQ

Activation-Aware Weight Quantization

- ▶ Protects top 1% salient weights from quantisation
- ▶ Calibration data identifies activation magnitudes
- ▶ 0.5–1% better perplexity than GPTQ at 4-bit
- ▶ Higher quality output; slightly slower to quantise
- ▶ Used in Hugging Face AutoAWQ library (pip install)

FP8 Native

H100 Transformer Engine — 2025 Default

- ▶ Hardware-native: zero dequantisation overhead
- ▶ 2× throughput vs BF16 on H100 SXM5
- ▶ Near-lossless quality (<0.5% perplexity delta)
- ▶ Supported in vLLM, TRT-LLM, SGLang, Ollama
- ▶ Meta uses FP8 for Llama 3 70B production serving

Decision Guide:

FP8 if serving on H100+. AWQ for highest quality at 4-bit on A10G/A100. GPTQ for broadest GPU compatibility & fastest quantisation speed.

[9] Frantar et al. (2022) GPTQ arXiv:2210.17323 [10] Lin et al. (2023) AWQ arXiv:2306.00978 [11] NVIDIA Transformer Engine Docs 2024

Low-Rank Adaptation (LoRA)

Efficient Fine-Tuning Through Parameter Decomposition

The Core Concept

Instead of updating all model parameters, inject small trainable matrices alongside frozen weights:

$$W_{\text{new}} = W_{\text{frozen}} + \Delta W$$

where $\Delta W = B \cdot A$ (low-rank decomposition)

Traditional Fine-Tuning

Pre-trained Model (7B parameters)

↓ **Update ALL** ↓

Fine-tuned Model (7B parameters)

✗ **7B × 4 bytes ≈ 28 GB per adapter**

LoRA Fine-Tuning

Pre-trained Model (Frozen, 7B params)

Matrix A (d×r)

Matrix B (r×d)

LoRA adapters: 8–16M params

✓ **Only 32–64 MB per adaptation!**

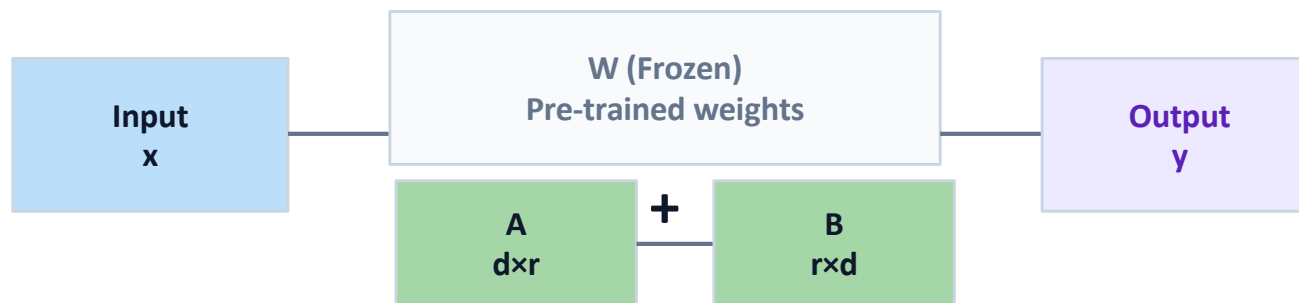
Why "Low-Rank"? ΔW is d×d (e.g. 4096×4096 = 16.7M params). LoRA decomposes as A (4096×r) and B (r×4096). At r=8: only 65K params — a 256× reduction. Rank r controls the adapter's capacity.

Ref: Hu et al. (2021) LoRA: Low-Rank Adaptation of Large Language Models arXiv:2106.09685

LoRA Architecture Visualization



How LoRA Adapters Integrate with Pre-trained Models



Frozen Path

- Original model weights
- No gradient updates
- Preserves pre-training knowledge

LoRA Path

- Trainable adapter matrices
- Gradient updates only here
- Task-specific learning at <1% params

Combined Output

- $y = W \cdot x + B \cdot A \cdot x$
- Sum of frozen + adapter paths
- Efficient: merged at inference time

Key Parameters

r (rank): 4–64. Controls adapter capacity. $r=8 \rightarrow 65K$ params per layer. **α (scaling):** Usually $\alpha=2r$. Scales LoRA contribution ($\Delta W = (\alpha/r) \cdot B \cdot A$).

Target layers: Q, V attention weights (and K, O, MLP for best results). **Memory:** <1% of full fine-tuning. Adapters merge into weights at zero inference cost.

QLoRA: Quantization + Fine-Tuning



Efficient Parameter-Efficient Training — 31× Memory Reduction

- ▶ **Base Model:** Quantized to 4-bit NF4 — frozen during training; double quantization saves additional ~0.4 GB per 7B
- ▶ **LoRA Adapters:** Small trainable matrices in BF16 — only these receive gradient updates
- ▶ **Memory Savings:** 70B model trainable on a single 48GB GPU (A6000/A40) — previously required 8× A100 80GB
- ▶ **Performance:** Matches full BF16 fine-tuning quality in most tasks; <1% delta on MMLU and MT-Bench

Memory Calculation

Full Fine-Tuning: $70\text{B} \times 4 \text{ bytes} \times 4 \text{ (model + grads + 2} \times \text{Adam states)} \approx 1,120 \text{ GB}$

QLoRA: $70\text{B} \times 0.5 \text{ bytes (NF4)} + \text{LoRA adapters (1 GB)} \approx 36 \text{ GB}$

31× Memory Reduction! (1,120 GB → 36 GB)

Ref: Dettmers et al. (2023) 'QLoRA: Efficient Finetuning of Quantized LLMs' arXiv:2305.14314

Quantization Decision Guide

Choosing the Right Precision for Your Scenario

Scenario	Recommended Precision	Rationale
Training from scratch	FP32 or BF16	Stability during gradient updates; BF16 preferred on H100 (1.5× faster, same range)
Fine-tuning large models	QLoRA (NF4 + LoRA)	31× memory reduction; 70B on 48GB single GPU; <1% quality loss
Production inference (cloud)	BF16 or FP8	BF16 on A100; FP8 on H100 for 2× throughput at same quality
Edge / on-device deployment	INT8 or NF4	Minimal memory; INT8 on most GPUs; NF4 for ARM/mobile
Real-time low-latency apps	INT8 or AWQ 4-bit	Fastest TTFT; AWQ 4-bit on A10G achieves 180+ tokens/sec for 7B

Rule of Thumb: For most production use cases, 4-bit quantization (GPTQ/AWQ) is sufficient with <2% quality loss. Use FP8 when H100 Tensor Cores are available. Only use BF16/FP16 when absolute maximum quality is required and VRAM permits.

Ref: Dettmers et al. GPTQ arXiv:2210.17323; Lin et al. AWQ arXiv:2306.00978; NVIDIA FP8 arXiv:2209.05433

GPU Memory Estimation



Understanding Memory Components — $M_{total} = M_{weights} + M_{KV} + M_{activation}$

Total Memory Formula

$$M_{total} = M_{weights} + M_{KV-cache} + M_{activation}$$

Model Weights

Fixed: $Parameters \times bytes_per_param$
7B BF16 = $7B \times 2 = 14$ GB
70B NF4 = $70B \times 0.5 = 35$ GB

KV-Cache

Dynamic: grows with seq length & batch
 $2 \times layers \times heads \times d_head \times seq \times batch \times 2$ bytes
Llama 3.3 8B @2K ctx = ≈ 1 GB

Activations

Temporary: during forward pass only
 $batch \times seq \times hidden \times layers \times 20$
Approx 10–20% of weights memory

Example: 7B model BF16 (14 GB) + KV cache (~ 2 GB) + activations (~ 1 GB) $\rightarrow$ needs 17 GB VRAM.

Ref: Rajbhandari et al. (2020) ZeRO: Memory Optimizations arXiv:1910.02054; Korthikanti et al. (2022) Reducing Activation Recomputation arXiv:2205.05198

Why Do We Need KV-Cache?



Understanding Attention Computation in Autoregressive Generation

The Inefficiency Problem: During text generation, transformers generate one token at a time. Each new token must attend to ALL previous tokens — recomputing K, V from scratch is $O(n^2)$ total work.

Without KV-Cache (Naïve)

Token 1: "The" → Compute Q_1, K_1, V_1

Token 2: "cat" → Compute $Q_1, K_1, V_1, Q_2, K_2, V_2$ (recompute!)

Token 3: "is" → Compute $Q_1, K_1, V_1, Q_2, K_2, V_2, Q_3, K_3, V_3$
Recompute all Q,K,V again

Token n: Recompute everything for tokens 1 to n-1

✗ Cost grows as $O(n^2)$ for n tokens!

With KV-Cache (Optimised)

Token 1: "The" → Compute K_1, V_1 → Cache them

Token 2: "cat" → Reuse K_1, V_1 ; compute K_2, V_2 → Cache

Token 3: "is" → Reuse K_1, K_2, V_1, V_2 ; compute K_3, V_3

Token n: Reuse all cached K,V — compute only K_n, V_n

✓ Only compute new K,V → $O(n)$ total!

Why Only Cache K and V?

Query (Q): Changes with each new token position — must recompute.

Keys (K) & Values (V): Represent previous context — constant once computed.
Cache them once, reuse forever.

KV-Cache: Technical Implementation



What Exactly is Stored and How Memory Grows

What's in the Cache?

For each layer: Store K and V matrices

For each head: Separate K, V vectors

For each token: One d_{head} -dim vector per head

Cache size/token = $2 \times \text{layers} \times \text{heads} \times d_{\text{head}} \times \text{bytes}$

Memory Growth Pattern

Prefill Phase: Process input prompt → build initial cache (fast, parallel)

Generation Phase: Add one K,V entry per generated token (sequential)

Memory: Linear in sequence length — grows with $\text{batch_size} \times \text{seq_len}$

Attention Computation with Cache

$Q_{\text{new}} = W_Q \times x_t$ # query for new token

$K_{\text{cached}} = [K_1, K_2, \dots, K_{t-1}]$ # retrieve from cache

$V_{\text{cached}} = [V_1, V_2, \dots, V_{t-1}]$ # retrieve from cache

$K_t = W_K \times x_t$;

$V_t = W_V \times x_t$ # compute for new token

$K_{\text{all}} = \text{concat}(K_{\text{cached}}, K_t)$ # append to cache

$\text{attention} = \text{softmax}(Q_{\text{new}} \times K_{\text{all}}^T / \sqrt{d_k}) \times V_{\text{all}}$

Trade-off

✓ **Speedup:**
10–100× faster generation (no recomputation)

✗ **Memory:**
Can exceed model weights for long contexts!

KV-Cache: The Hidden Memory Consumer



Understanding and Quantifying Key-Value Cache Overhead

KV-Cache Memory Formula

$$M_{KV} = 2 \times \text{layers} \times \text{heads} \times d_{\text{head}} \times \text{seq_len} \times \text{batch_size} \times \text{bytes_per_element}$$

Example: Llama 3.3 8B

- ▶ **Layers:** 32
- ▶ **Attention Heads:** 32
- ▶ **d_head:** 128
- ▶ **Precision:** BF16 (2 bytes)

For 2K context, batch=1:

$$M_{KV} = 2 \times 32 \times 32 \times 128 \times 2048 \times 1 \times 2$$
$$= 536,870,912 \text{ bytes} \approx 0.5 \text{ GB}$$

≈ 1 GB

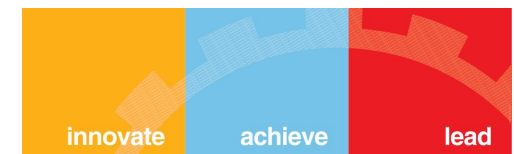
(exact: 0.54 GB — model reports ~1 GB with overhead)

Scaling Warning:

At 128K context (batch=8), same model: $M_{KV} = 2 \times 32 \times 32 \times 128 \times 131,072 \times 8 \times 2 \approx 137 \text{ GB}$ — more than the model weights themselves (16 GB). This is why PagedAttention and GQA (Grouped-Query Attention) are essential at scale.

Ref: Kwon et al. (2023) Efficient Memory Management with PagedAttention (vLLM) arXiv:2309.06180; Ainslie et al. (2023) GQA arXiv:2305.13245

Hands-On: Memory Calculation



Calculate VRAM for a 175B Model (Claude 3.5 Sonnet-class) Inference

Given Configuration:

- ▶ **Parameters:** 175B
- ▶ **Precision:** BF16 (2 bytes/param)
- ▶ **Sequence Length:** 4,096 tokens
- ▶ **Batch Size:** 4
- ▶ **Layers / Heads / d_head:** 96 / 96 / 128

1. Model Weights

$175B \times 2 \text{ bytes}$

= 350 GB

2 (for Key and Value matrices)

$\times 96$ (Layers)

$\times 96$ (Heads)

2. KV-Cache

$2 \times 96 \times 96 \times 128 \times 4096 \times 4 \times 2$

$\approx 96 \text{ GB}$

$\times 128$ (Dimension per head)

$\times 4096$ (Sequence Length)

$\times 4$ (Batch Size)

$\times 2$ (Bytes per element - BF16)

3. Activations

$\text{batch} \times \text{seq} \times \text{hidden} \times \text{layers} \times 2$

$4 * 4096 * 12,288 * 96 * 2 \text{ Bytes} \approx 38 \text{ GB}$

Total VRAM Required

$350 + 96 + 38 \approx 484 \text{ GB}$



Practical GPU Setup:

- 6–7× A100 40GB (with tensor parallelism)
- 2× H100 80GB (recommended, FP8 cuts to ~250 GB)

With FP8 quantization: weights shrink to ~175 GB → total $\approx 309 \text{ GB}$ → feasible on 4× H100 80GB.

Memory-Efficient Inference at Scale



Advanced Techniques for Production Deployment — 2025

FlashAttention-3

- Tiles attention computation to fit in SRAM
- 3–5× faster than FA-2 on H100/H200
- 10–20× memory reduction vs standard attention
 - Essential for 8K+ contexts; 1M+ context on H100
 - Supports FP8 natively — now production default

PagedAttention (vLLM)

- Virtual memory paging for KV-cache blocks
 - Reduces wasted GPU memory from 60% → <10%
 - 2–5× higher serving throughput
 - Enables continuous batching (no padding waste)
 - Used by vLLM, TGI, TRT-LLM in production

CPU / NVMe Offloading

- KV-cache spills to CPU DDR5 (10× cheaper)
- NVMe offload for 10× longer contexts
- Trade-off: 2–3× slower generation latency
 - FlexGen: 100B model on single consumer GPU
 - Best for research / low-throughput workloads

Parallelism Strategies

Tensor Parallelism (TP)

- Split weight matrices across GPUs (Megatron-LM style)
- Lower latency — all-reduce per layer
- Best for 2–8 GPUs; adds ~5% communication overhead

Pipeline Parallelism (PP)

- Split transformer layers across GPUs (depth-wise)
- Higher throughput via micro-batching
- Best for 8+ GPUs; ~15% bubble overhead without scheduling

🎯 **Recommendation: FlashAttention-3 + PagedAttention as baseline. Add Tensor Parallelism for 70B+ models (TP=4 on 4× H100 is optimal).**

Ref: Shah et al. (2024) FlashAttention-3 arXiv:2407.08608; Kwon et al. (2023) vLLM PagedAttention arXiv:2309.06180

Token Economics: Understanding API Costs

2025 Pricing Landscape — per 1M tokens (indicative)

Model	Input	Output	Cached Input	Context
GPT-4o (OpenAI)	\$5	\$15	\$2.50	128K
GPT-4 Turbo (OpenAI)	\$10	\$30	\$5.00	128K
Claude 3.7 Sonnet (Anthropic)	\$3	\$15	\$0.30	200K
Claude 3.5 Haiku (Anthropic)	\$0.80	\$4	\$0.08	200K
Gemini 2.0 Flash (Google)	\$0.075	\$0.30	\$0.019	1M
DeepSeek-V3 (DeepSeek API)	\$0.27	\$1.10	\$0.014	128K
Mixtral 8×7B (Mistral API)	\$0.60	\$0.60	N/A	32K
Self-Hosted vLLM (cloud GPU)	\$0.10–0.30*	\$0.10–0.30*	N/A	Model-dep.

Key Observation: Output tokens cost 3–10× more than input. Minimize generation length, use structured outputs, and maximize prompt caching to reduce cost by 70–90%.

Prices as of May 2025. Always verify at openai.com/pricing, anthropic.com/pricing, ai.google.dev. *Self-hosted cost = GPU compute only, excludes DevOps overhead.



Prompt Caching: 90% Cost Reduction

Reuse Repeated Context Across Requests

How It Works

- ▶ **Cache static prompts:** System prompts, few-shot examples, large documents
- ▶ **TTL:** 5-minute window (Anthropic); 1 hour (OpenAI); refreshes on hit
- ▶ **Minimum tokens:** 1,024 tokens for Claude; 512 tokens for OpenAI GPT-4o
- ▶ **Discount:** 90% on cached input tokens (Anthropic); 50% (OpenAI)

Cost Example

Without Caching:

10K tokens × 100 requests = 1M tokens × \$3 = \$3.00

With Caching:

First request: 10K × \$3/1M = \$0.030

Next 99: 10K × \$0.30/1M × 99 = \$0.297

Total: \$0.327 → 89% saving!

Optimisation Strategies

1. Prompt Structure:

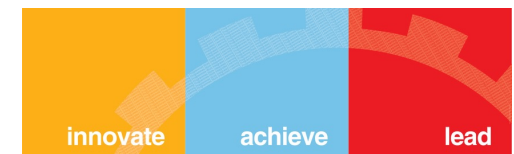
Place static content first (system prompt, examples, docs), dynamic query last. Any change before the cache breakpoint invalidates the entire cache.

2. Cache Invalidation:

Any content change = cache miss. Use fixed templates. Version your system prompts; avoid timestamps or dynamic injections before the query turn.

Ref: Anthropic Prompt Caching Docs: docs.anthropic.com/en/docs/build-with-claude/prompt-caching; OpenAI: platform.openai.com/docs/guides/prompt-caching

Production Cost Optimization Playbook



From \$12K/month to \$2K/month — Real Strategies That Work

Immediate Actions (Week 1)

- ▶ Enable prompt caching on all endpoints with repeated context (system prompts >1K tokens)
- ▶ Route simple tasks (intent, NER, classification) to SLMs — expect 70–90% cost drop on that slice
- ▶ Set max_tokens appropriately — open-ended max_tokens = output billing at ceiling
- ▶ Batch non-urgent requests (OpenAI Batch API = 50% discount; Anthropic bulk API)

Advanced Techniques (Month 1)

- ▶ Model routing by complexity (RouteLLM / custom classifier) — target 80/15/5 SLM/MoE/Dense split
- ▶ Self-host quantized models (vLLM + AWQ) for high-volume workloads (>5B tokens/month)
- ▶ Speculative decoding: draft model (SLM) + target model → 2× generation speed at same cost
- ▶ Efficient RAG: top-3 instead of top-10 chunks reduces context by 60–70%; use re-rankers

Real-World Case Study

Production RAG system: \$12K/month → \$2K/month using:
50% prompt caching reduction + Gemma-2B router → Claude (80/20 split) + 4K max context vs 16K + AWQ-4bit self-hosted Mixtral for medium tasks

Ref: Ong et al. (2024) RouteLLM arXiv:2406.18665; Chen et al. (2023) Speculative Decoding arXiv:2302.01318

Emerging Trends in LLM Optimization (2025-26)



What's Next in Model Efficiency — Research to Production Pipeline

1-BIT LLMs (BitNet b1.58+)

- Ternary quantization now mainstream — weights $\{-1, 0, +1\}$
- 0.5-BIT exploration underway with $<0.5\%$ quality loss
- 2-5x energy efficiency vs. FP8/INT8
- MMLU parity achieved at 8B+ scale
- Status: Production deployment in resource-constrained environments

Mixture of Depths (MoD) v2

- Dynamic computation integrated with token skipping
- Compatibility with State Space Models (Mamba hybrid)
- 60–70% FLOP reduction with $<0.3\%$ quality drop
- Composable with LoRA and other adapters
- Adopted in Gemma 4, next-gen Llama (2026)

FlashAttention-4

- Hardware-optimized kernels for H200, Grace, B200 GPUs
- 3-5x faster than FlashAttention-3 on new hardware
- Native support for grouped query attention (GQA)
- Multi-GPU ring attention for $>1\text{M}$ context windows
- Now standard in vLLM, TRT-LLM, JAX ecosystem

Hardware-Aware Quantization (2.0)

- Dynamic quantization formats optimized per hardware
- NVIDIA NVQ (mixed INT4/INT8) on new GPU architectures
- Mobile quantization support — INT4 inference on Apple Neural Engine
- Auto-format selection via hardware profiling
- 3-4x better GPU utilization vs. static precision

Ref: Emerging 2026 trends in model optimization based on 2025 trajectories — quantization, dynamic computation, and hardware co-design convergence.



Self-Hosting vs. API: Break-Even Analysis

Making the Right Infrastructure Decision — 2025 Cost Benchmarks

Factor	Managed API	Self-Hosted (vLLM + H100)
Upfront Cost	\$0	\$5K–50K (GPU lease/purchase) + \$10K–30K setup
Monthly @ 1B tokens	~\$3,000 (Claude 3.5) / ~\$300 (DeepSeek API)	~\$800–1,200 (H100 SXM5 spot) + \$8K–12K DevOps
Maintenance	Zero — SLAs, updates, scaling included	\$8K–12K/month DevOps + on-call engineering
Scalability	Instant auto-scaling (seconds)	Hours to days — GPU provisioning + deployment
Data Privacy	Shared infrastructure — data sent to vendor	Full control — on-premises or private VPC
Latency	Variable (P50: 200–500ms for 7B models)	Predictable (P50: 50–150ms for 7B on H100)
Break-Even Point	< 5B tokens/month → API wins	≥ 10B tokens/month → self-hosting saves 60–70%

Decision Rule: Start with APIs (<5B tok/mo). Consider self-hosting at 5–10B tok/mo with quantised Llama/Qwen. Self-host only at >10B tok/mo for clear ROI.

Costs as of May 2025 — GPU spot prices vary significantly. Model: H100 SXM5 80GB on AWS p5.48xlarge (\$98.32/hr for 8× H100). Verify current pricing before infrastructure decisions.

Key Takeaways



What every ML engineer and architect must internalise from this lecture



Architecture Shapes Cost

Dense (all params active) → MoE (94% compute saving in DeepSeek-V3) → SLM (80% of queries at <5% cost). Route by task complexity.



Quantization is Non-Negotiable

NF4/AWQ 4-bit = 4× memory, ~98% quality. FP8 on H100 = 2× throughput at near-lossless quality. BF16 on A100 is the minimum for inference.



SSMs Unlock 1M+ Context

Mamba-2 is $O(n)$ not $O(n^2)$. Jamba-1.5 (hybrid) handles 256K context in production. Choose SSM/hybrid when documents exceed 128K tokens.



KV-Cache Dominates Memory at Scale

At 128K context, KV-cache can exceed model weight memory. Use GQA, PagedAttention, and FlashAttention-3 as production baseline.



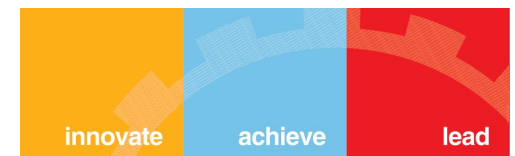
Prompt Caching = Immediate 89% Saving

90% token discount (Anthropic), 50% (OpenAI). Always place static context first, dynamic query last. Minimum 1,024 tokens for Claude.



Self-Host Only Beyond 10B Tokens/Month

API wins at <5B tok/mo. Self-host with quantized vLLM saves 60–70% at scale but adds \$8–12K/month DevOps burden — budget for it.



References

1. <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state>
2. https://buildml.substack.com/p/top-24-llm-questions-asked-at-deepmind?utm_source=publication-search

References & Further Reading



- [1] Achiam et al. (2023). GPT-4 Technical Report. OpenAI. arXiv:2303.08774
- [2] Anthropic. (2025). Claude 3.7 Sonnet Model Card. anthropic.com, March 2025
- [3] Meta AI. (2024). Llama 3 Technical Report. arXiv:2407.21783
- [4] Google DeepMind. (2025). Gemini 2.0 Technical Report. deepmind.google
- [5] DeepSeek-AI. (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437
- [6] Jiang et al. (2024). Mixtral of Experts. Mistral AI. arXiv:2401.04088
- [7] Qwen Team. (2024). Qwen2 Technical Report. Alibaba Cloud. arXiv:2407.10671
- [8] AI21 Labs. (2024). Jamba-1.5 Technical Report. ai21.com, August 2024
- [9] Frantar et al. (2022). GPTQ: Accurate Post-Training Quantization. arXiv:2210.17323
- [10] Lin et al. (2023). AWQ: Activation-aware Weight Quantization. arXiv:2306.00978
- [11] Micikevicius et al. (2022). FP8 Formats for Deep Learning. arXiv:2209.05433
- [12] Gu & Dao. (2023). Mamba: Linear-Time Sequence Modeling. arXiv:2312.00752
- [13] Dettmers et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314
- [14] Hu et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685
- [15] Kwon et al. (2023). Efficient Memory Management with PagedAttention (vLLM). arXiv:2309.06180
- [16] Shah et al. (2024). FlashAttention-3: Fast and Accurate Attention with Asynchrony. arXiv:2407.08608
- [17] Dao et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention. arXiv:2205.14135
- [18] Ma & Shen. (2024). The Era of 1-bit LLMs: All Models in 1.58 Bits. arXiv:2402.17764
- [19] Ong et al. (2024). RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665
- [20] Vaswani et al. (2017). Attention Is All You Need. NeurIPS. arXiv:1706.03762

Additional: lmsys.org/chat (live benchmarks) · huggingface.co/spaces/open-llm-leaderboard · vllm.readthedocs.io · docs.anthropic.com